
目次

はじめに	001
------	-----

第 NEST 章

【Rails7+esbuild】必要な JavaScript — Takahashi	001
のみ個別に読み込みたい時	
Cloudflare で独自ドメインを取得する — tanuki	002
未経験からフルリモートのエンジニアに — midori	003
なってもうすぐ1年なのでフルリモ向い てる人を考えてみる	

第 HUB 章

ChatGPT の効果的な使い方 — Tanaka	001
Sampe Title — Taro	002

第 自由章

ChatGPT の効果的な使い方 — Tanaka 001

SamIpe Title — Taro 002

Javascriptからの護身術 — ぼとふね 003

おわりに 004

奥付 005

はじめに

この同人誌『RUN-NEST Hub』は、技術を愛する開発者たちによる自由な寄稿によって構成されています。それぞれの筆者が日々取り組んでいるテーマや技術的な挑戦を、熱量のこもった言葉で綴っていただきました。

この誌面が、あなたの開発のヒントや新たな視点となれば幸いです。

みたいなことを未来の私が書く
よ~~~~~

Vivliostyle + Markdown ドキュメントガイド

基本的な Markdown 記法

見出し

```
# 見出し1 (h1)
## 見出し2 (h2)
### 見出し3 (h3)
```

見出し1 (h1)

見出し2 (h2)

見出し3 (h3)

段落・改行

段落は空行で区切ります。

改行は行末に半角スペースを2つ入れます。

強調

```
*斜体* または _斜体_
**太字** または __太字__
```

斜体 または 斜体

太字 または 太字

リスト

- 簡条書き
- 2つ目
 - ネストされたリスト

- ```
1. 番号付きリスト
2. 2つ目
```

- 箇条書き
- 2つ目
  - ネストされたリスト

- ```
1. 番号付きリスト
2. 2つ目
```

リンク

PDF 配布時にテキストにリンクを付けたい場合は以下のように記述します。

```
[リンクテキスト](https://example.com)
```

リンクテキスト

但し、著作物の引用などを行う場合は基本的に注釈を用いてください。

注釈

```
これは注釈付きの文章です[^1]。
```

```
[^1]: これは脚注の内容です。
```

これは注釈付きの文章です¹。

画像

画像をimagesフォルダに追加するか、URLリンクを取得します。

```
![説明文](../images/image.png)
```



説明文

画像の幅を指定するには次のように記述します：

```
![説明文](../images/midori.png){style="width:50%;"} 
```



説明文

コード

インラインコード：

```
これは `コード` です。
```

これは `コード` です。

コードブロック：

```
```\nconsole.log("Hello Vivliostyle!");\n```
```

```
console.log("Hello Vivliostyle!");
```

## HTMLとの併用

VivliostyleではHTMLも使用可能です：

## 改ページ

`class="page-break"` を付与する

```
<div class="page-break">ここで改行</div>
```

ここで改行

## カスタムスタイルを当てたい

CSSを当てたい箇所にクラスを付与する

```
<div class="custom-box">
 カスタムスタイルのボックス
</div>
```

CSSを用いて装飾：

```
.custom-box {
 border: 1px solid #ccc;
 padding: 1em;
 background-color: #f9f9f9;
}
```

## 参考リンク

- [Vivliostyle 公式サイト](#)
- [VFM仕様ドキュメント](#)
- [Vivliostyle CLI](#)

---

1. これは脚注の内容です。

## 【Rails7+esbuild】必要なJavaScriptのみ個別に読み込みたい時

著者名

### 必要なJavaScriptのみ個別に読み込みたい

方法が分からず、とりあえず動きやいい精神でapplication.jsに全部読み込ませてたら、パフォーマンス診断で怒られてしまった（あたり前田のクラッカー）



パフォーマンス ユーザー補助



SEO



PWA



### パフォーマンス

推定値のため変動する可能性があります。[パフォーマンススコアの計算](#)は、これらの指標を基に行っています。[計算ツールはこちら。](#)

▲ 0-49

■ 50-89

● 90-100

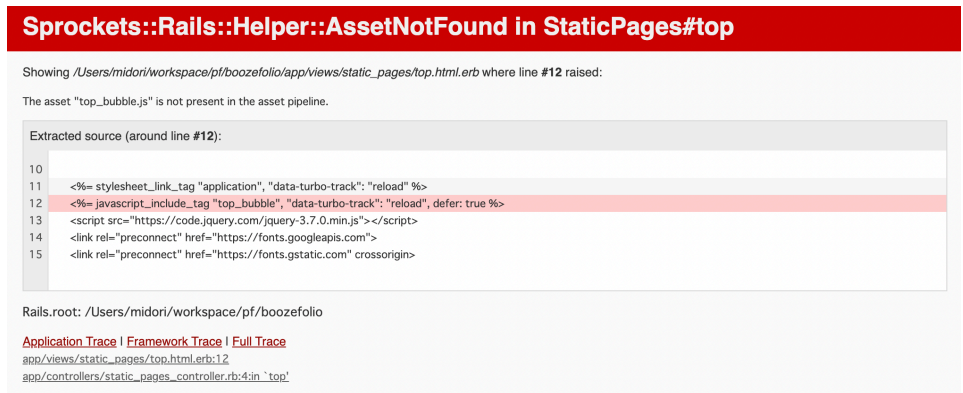


スクリーンショット 2023-08-24 10:36:39.png

不必要なjsを読み込んでいるページでブラウザエラーも出ていたので、これを機に直すことに。

# そもそも、なぜ application.js に詰め込んだ？

**Sprockets::Rails::Helper::AssetNotFound in ○○** のエラーになる。



スクリーンショット 2023-08-24 10.41.42.png

ここでは、top\_bubble.js というファイルを読み込ませようとしています、上記エラーでうまくいかず。

**The asset "top\_bubble.js" is not present in the asset pipeline.**

「"top\_bubble.js" がアセット・パイプラインに存在しません。」

とあることから、asset 周りなのはわかるのですが、どうしたら良いのやら。。

初学者のため、間違いやまずいやり方があればぜひコメントでご指摘ください 🙏

## 確認したこと

1. パスが間違っていないか

2. app/assets/javascripts の設定

3. app/assets/builds に個別に読み込みたいjsが buildされているか

1と2は結果的には関係がなかったのですが、忘備録に簡単に記載しておきます。<sup>1</sup>

上記記事によると Rails 6 までは app/assets/javascripts 以下の全てのファイルを自動で読み込んでいたそうです。その設定ファイルが config/initializers/assets.rb とのこと。

ただし、今回はRails7を使用していることから、そもそも `app/javascript` にあるのでファイル構成が違います。ものは試しなので下記のように設定を記載しましたが、結果として3.だけで動いたため関係ありませんでした。23

```
Be sure to restart your server when you modify this file.

Version of your assets, change this if you want to expire all your asset...
Rails.application.config.assets.version = '1.0'

Add additional assets to the asset load path.
Rails.application.config.assets.paths << Emoji.images_path

Precompile additional assets.
application.js, application.css, and all non-JS/CSS in the app/assets
folder are already added.
Rails.application.config.assets.precompile += %w(admin.js admin.css)

Rails.application.config.assets.precompile += %w(custom/*.js)
#追加してみたけど関係なかった
#今回個別に読み込みたいjsファイルは全てapp/javascript/custom以下に置いているため...
```

`config/initializers/assets.rb`はいつ使うのか誰か教えてください🙏

`assets.rb`についてはプリコンパイルするファイルを指定する時に必要でした。デプロイ時にエラーが出たので、こちらをご覧ください。

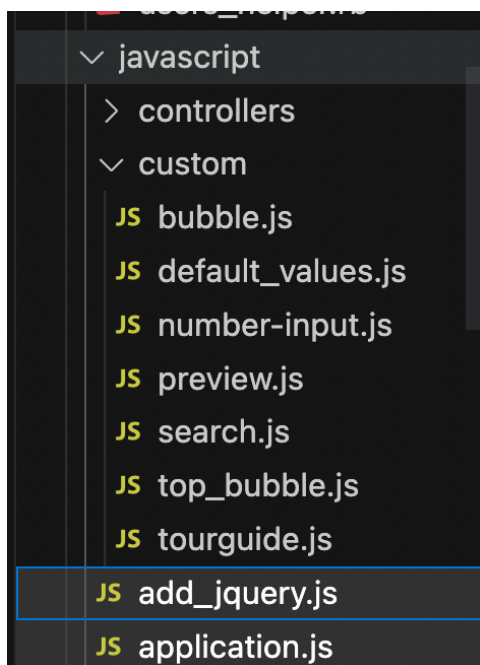
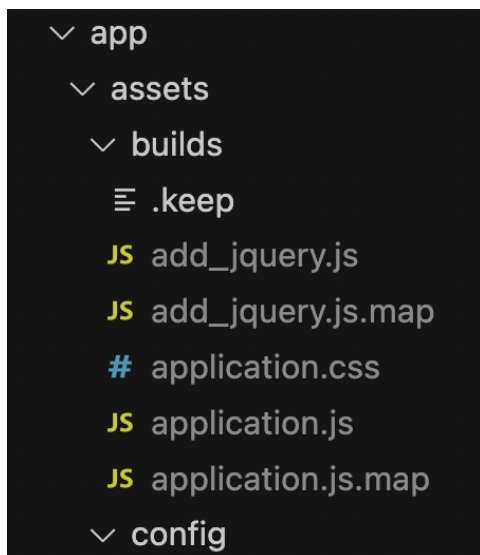
<https://qiita.com/MIDO-ruby7/items/9718b40c97e4d282ae83>

### 3.app/assets/buildsに個別に読み込みたいjsがbuildされているか

esbuildでは `app/javascript` 以下のファイルが自動で `app/assets/builds` にコンパイルされます。

※コンパイル：人間が書いたコードをコンピュータの処理装置であるCPUが解釈して実行できる形に変換すること。俺たちにとってのDeepleみたいなもんだろ（違）。



`app/javascript``app/assets/builds`

ブラウザは、この `app/assets/builds` にコンパイルされたコードを読み取るので、そもそもここにファイルがないと The asset "top\_bubble.js" is not present in the asset pipeline.と言われるわけです。

esbuildの場合、どこにコンパイルのためのコマンドがあるかというと、`package.json`

```
{
 "name": "app",
 "private": "true",
 "dependencies": {
 "@hotwired/stimulus": "^3.2.1",
 "esbuild": "^0.17.19",
 省略
 "tailwindcss": "^3.3.2",
 "trix": "^2.0.5"
 },
 "scripts": {
 "build": "esbuild app/javascript/*.js --bundle --sourcemap --outdir=app...
// ↑ここでapp/javascript/*.js をassets/builds にコンパイルするよう要求している
 "build:css": "tailwindcss -i ./app/assets/stylesheets/application.tail...
 }
}
```

ということでコードを追加。

```
"scripts": {
 "build": "esbuild app/javascript/*.js --bundle --sourcemap --outdir=app...
 "build:css": "tailwindcss -i ./app/assets/stylesheets/application.tail...
 "build:custom": "esbuild app/javascript/custom/*.js --bundle --sourcemap...
// ↑これを追加
}
```

\$ bin/dev コマンドで再起動したのですが、yarn コマンド(npm 使ってる人は npm)を打たないと自動では build されないようです。

```
$ yarn build:custom
```

```
boozefolio % yarn build:custom
yarn run v1.22.19
$ esbuild app/javascript/custom/*.js --bundle --sourcemap --outdir=app/ass...

app/assets/builds/bubble.js 254.5kb
app/assets/builds/top_bubble.js 254.4kb
app/assets/builds/tourguide.js 2.4kb
app/assets/builds/search.js 945b
app/assets/builds/default_values.js 927b
app/assets/builds/preview.js 610b
app/assets/builds/number-input.js 390b
app/assets/builds/bubble.js.map 466.1kb
app/assets/builds/top_bubble.js.map 466.1kb
app/assets/builds/tourguide.js.map 3.1kb
...and 4 more output files...

🌟 Done in 0.23s.
```

app/assets/builds に無事コンパイルされました。

```

 ▼ app
 ▼ assets
 ▼ builds
 ≡ .keep
 JS add_jquery.js
 JS add_jquery.js.map
 # application.css
 JS application.js
 JS application.js.map
 JS bubble.js
 JS bubble.js.map
 JS default_values.js
 JS default_values.js.map
 JS number-input.js
 JS number-input.js.map
 JS preview.js
 JS preview.js.map
 JS search.js
 JS search.js.map
 JS top_bubble.js
 JS top_bubble.js.map
 JS tourguide.js
 JS tourguide.js.map

```

これを読み込みたいviewファイルで読み込んでやればOK

例 : `<%= javascript_include_tag "top_bubble" %>`

## その他詰まったところ

jQueryなどを使用している際は、個別にimportする必要があります。

```

import "../add_jquery" //コードを追加

var items = 30;

for (var i = 0; i <= items; i++) {
 var moveVal = Math.ceil(Math.random() * 50);
 var posVal = Math.ceil(Math.random() * 50);

```

ただ、なぜかjQueryをCDNで読み込まないとimportできず。。。application.jsでは大丈夫だったのになぜなのでしょう？ むむむ。

```

<%= javascript_include_tag "top_bubble" %>
<script src="https://code.jquery.com/jquery-3.7.0.min.js"></script>
// CDNを追加

```

このままではデプロイ時にエラーが出ました。

詳しくは別記事にまとめたので下記をご覧ください。4

---

<https://keruuweb.com/rails->

[%E5%BF%85%E8%A6%81%E3%81%AAjavascript%E3%81%AE%E3%81%BF%E3%82%92%](https://keruuweb.com/rails-%E5%BF%85%E8%A6%81%E3%81%AAjavascript%E3%81%AE%E3%81%BF%E3%82%92%E8%AA%AD%E3%81%BF%E8%BE%BC%E3%82%80/)

1. [E8%AA%AD%E3%81%BF%E8%BE%BC%E3%82%80/](https://keruuweb.com/rails-%E5%BF%85%E8%A6%81%E3%81%AAjavascript%E3%81%AE%E3%81%BF%E3%82%92%E8%AA%AD%E3%81%BF%E8%BE%BC%E3%82%80/)

↵

[https://railsguides.jp/asset\\_pipeline.html#%E5%88%A5%E3%81%AE%E3%83%A9%E3%82](https://railsguides.jp/asset_pipeline.html#%E5%88%A5%E3%81%AE%E3%83%A9%E3%82%A4%E3%83%96%E3%83%A9%E3%83%AA%E3%82%92%E4%BD%BF%E3%81%86)

2. [%A4%E3%83%96%E3%83%A9%E3%83%AA%E3%82%92%E4%BD%BF%E3%81%86](https://railsguides.jp/asset_pipeline.html#%E5%88%A5%E3%81%AE%E3%83%A9%E3%82%A4%E3%83%96%E3%83%A9%E3%83%AA%E3%82%92%E4%BD%BF%E3%81%86)

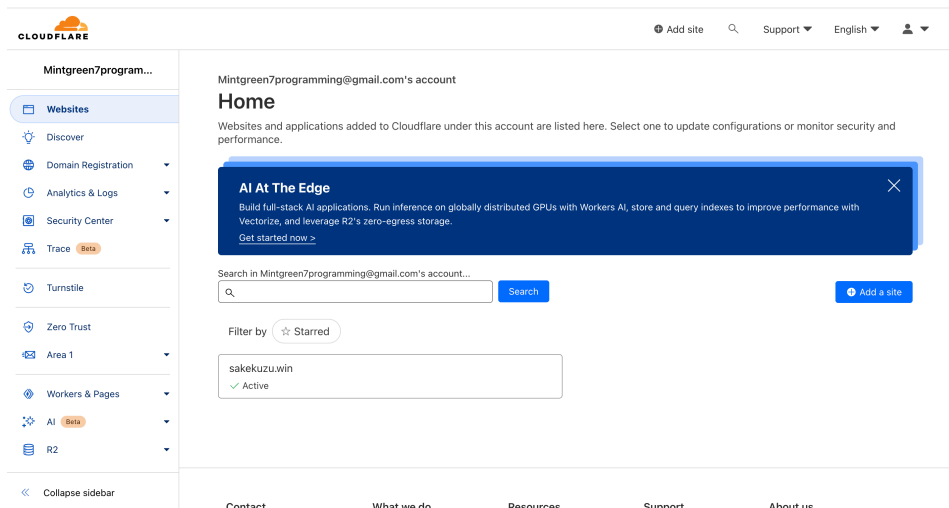
↵

3. [https://www.wantedly.com/companies/wantedly/post\\_articles/354873](https://www.wantedly.com/companies/wantedly/post_articles/354873)↵

# Cloudflareで独自ドメインを取得する

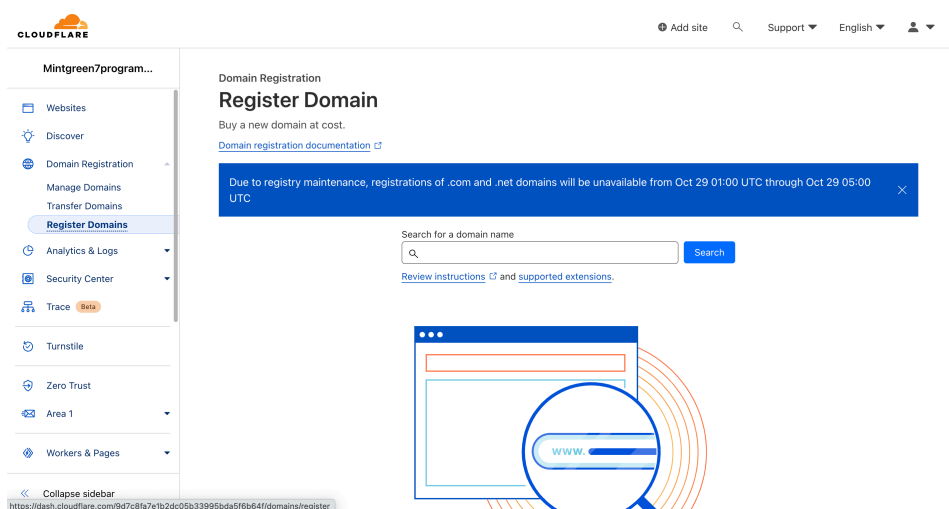
## ①Cloudflare に登録・ログイン

右上のプルダウンから日本語にも変えられます。

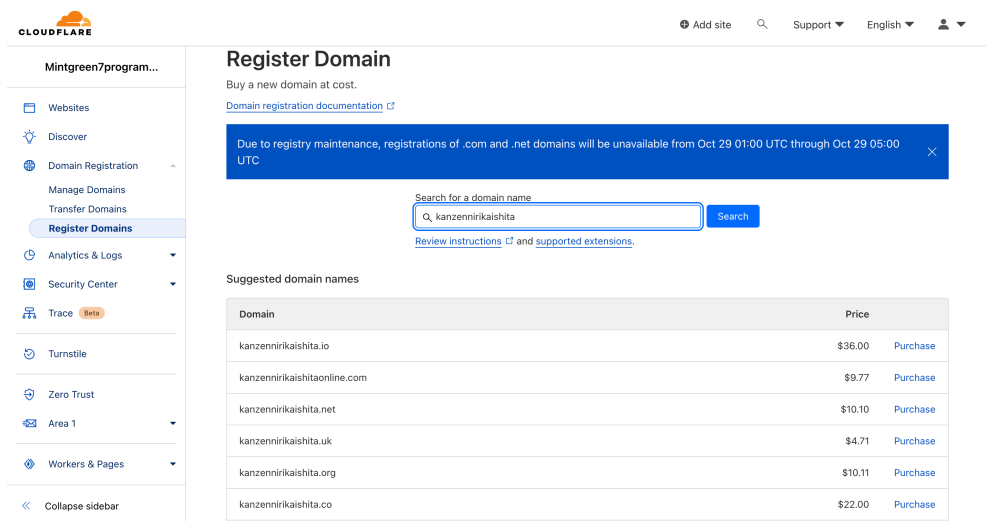


## ②Domain Registration > Register Domain

左のメニューからRegister Domainを選択し、



検索バーから、取得したいドメイン名を検索



**Register Domain**  
Buy a new domain at cost.  
[Domain registration documentation](#)

Due to registry maintenance, registrations of .com and .net domains will be unavailable from Oct 29 01:00 UTC through Oct 29 05:00 UTC

Search for a domain name  
Q kanzennirikaishita Search

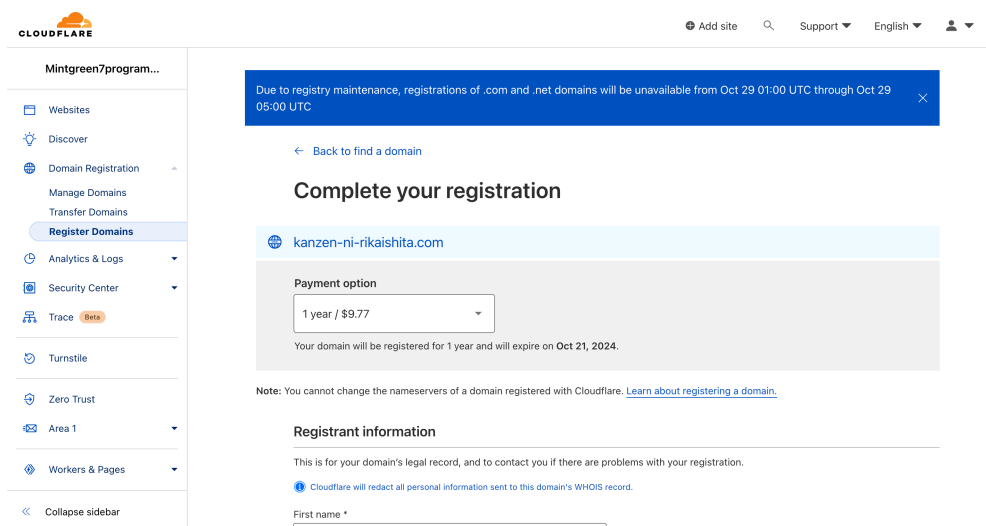
[Review instructions](#) and [supported extensions](#).

Suggested domain names

Domain	Price
kanzennirikaishita.io	\$36.00 Purchase
kanzennirikaishitaonline.com	\$9.77 Purchase
kanzennirikaishita.net	\$10.10 Purchase
kanzennirikaishita.uk	\$4.71 Purchase
kanzennirikaishita.org	\$10.11 Purchase
kanzennirikaishita.co	\$22.00 Purchase

スクリーンショット 2023-10-21 21.13.03.png

### ③取得したいドメイン名を選択し、フォームに情報を入力して購入



Due to registry maintenance, registrations of .com and .net domains will be unavailable from Oct 29 01:00 UTC through Oct 29 05:00 UTC

[Back to find a domain](#)

**Complete your registration**

kanzen-ni-rikaishita.com

Payment option  
1 year / \$9.77

Your domain will be registered for 1 year and will expire on Oct 21, 2024.

Note: You cannot change the nameservers of a domain registered with Cloudflare. [Learn about registering a domain](#).

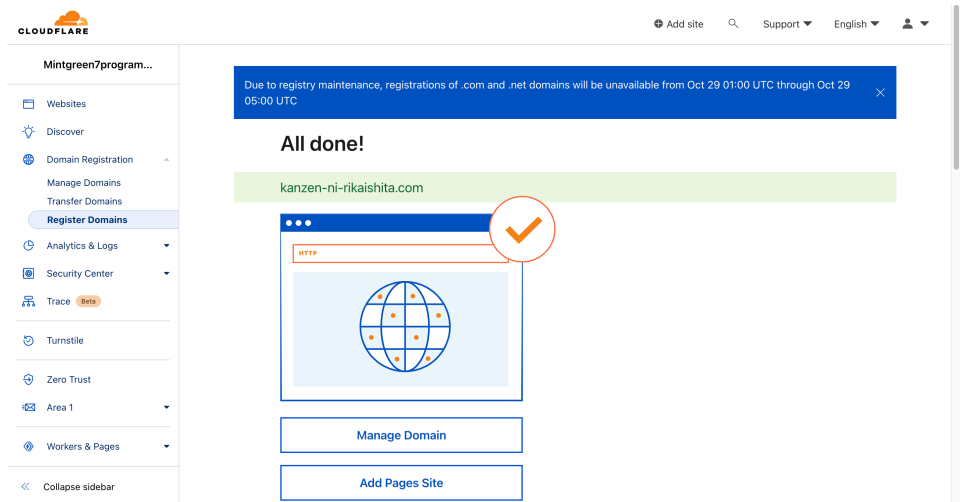
**Registrant information**

This is for your domain's legal record, and to contact you if there are problems with your registration.

Cloudflare will redact all personal information sent to this domain's WHOIS record.

First name \*

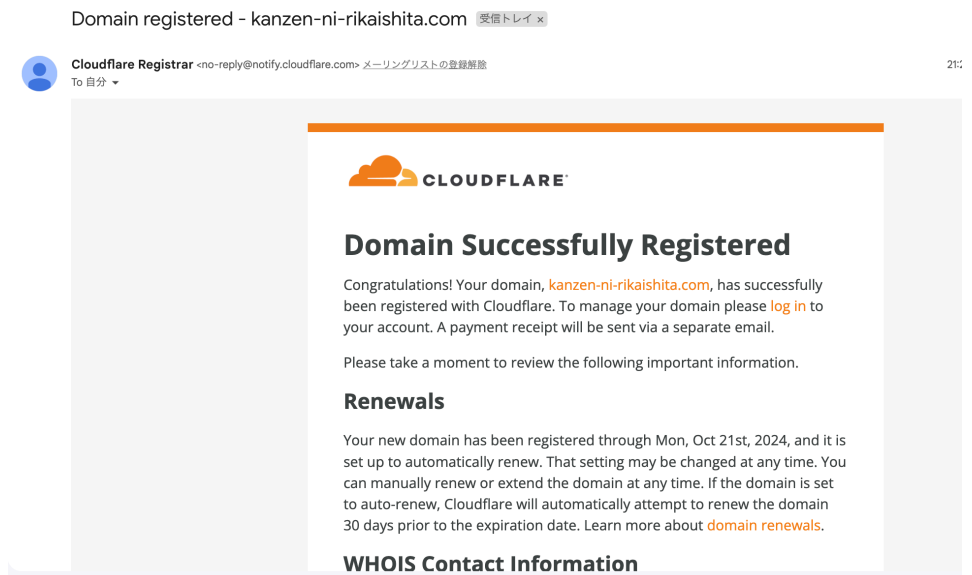
スクリーンショット 2023-10-21 21.22.35.png



スクリーンショット 2023-10-21 21.23.12.png

楽天カードはなぜか使えません 🤔

ドメイン取得が完了したら以下のようなメールが来る



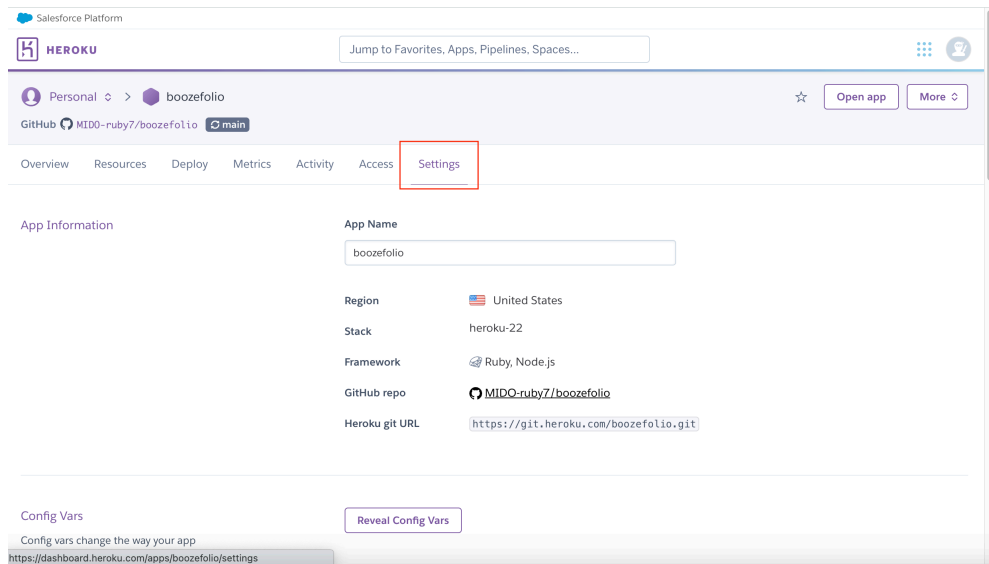
スクリーンショット 2023-10-21 21.40.35.png

ドメインが自動更新に設定されている場合、Cloudflareは有効期限の30日前に自動的にドメインが更新される

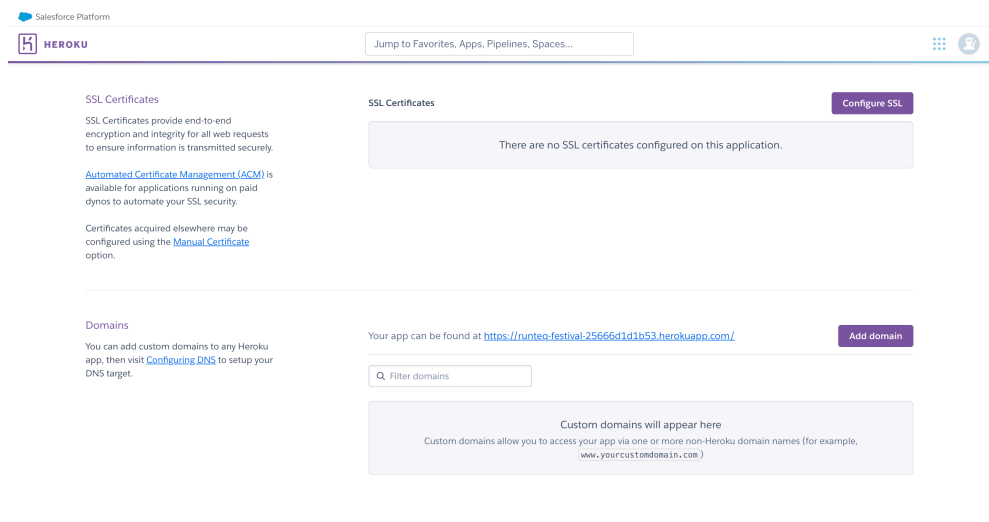
## 取得した独自ドメインを heroku で設定する

### ① heroku 側の設定を行う

独自ドメインを設定したいアプリの setting 画面に遷移する



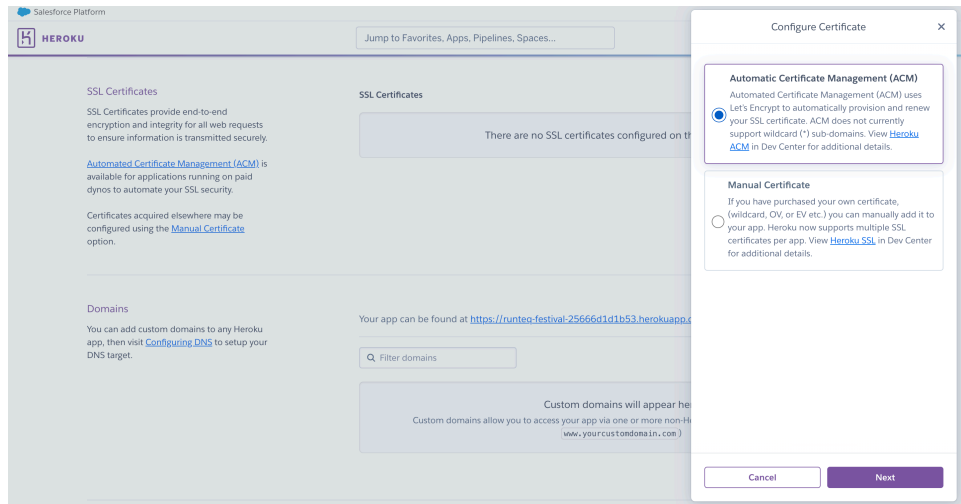
下にスクロールすると、"SSL Certificates"と"Domains"がありこの2つを設定していく





## SSL Certificatesを設定する

まず、「Configure SSL」ボタンを押し、「Automatic Certificate Management (ACM)」を選択して「Next」を押す。



スクリーンショット 2023-10-21 21.52.44.png

ACMは、TLS/SSL サーバ証明書を heroku 側で自動で自動的に管理してくれる設定。

【補足：SSLサーバ証明書って？】

SSLサーバ証明書をWebサイトに設定することによって、「なりすましの防止」、やり取りされるデータの「改ざん検知」、「盗聴防止」、「中間者攻撃による通信の改ざんを検知」といった効果が得られます。アクセスしているページにSSLサーバ証明書の設定がされており、SSLによって暗号化保護がされているかを確認する手段は、ブラウザのアドレスが「https://」となっているかどうかで判断できます。

[https://www.jipdec.or.jp/library/report/ssl\\_cert.html](https://www.jipdec.or.jp/library/report/ssl_cert.html)

"Your certificate is automatically managed. Add a custom domain to your app."と出ていればOK

#### SSL Certificates

SSL Certificates provide end-to-end encryption and integrity for all web requests to ensure information is transmitted securely.

[Automated Certificate Management \(ACM\)](#) is available for applications running on paid dynos to automate your SSL security.

Certificates acquired elsewhere may be configured using the [Manual Certificate](#) option.

#### SSL Certificates

[Configure SSL](#)

Your certificate is automatically managed. Add a custom domain to your app.

#### Domains

You can add custom domains to any Heroku app, then visit [Configuring DNS](#) to setup your DNS target.

Your app can be found at <https://runteg-festival-25666d1d1b53.herokuapp.com/>

[Add domain](#)[All Domains](#)[Refresh ACM Status](#)

Custom domains will appear here

Custom domains allow you to access your app via one or more non-Heroku domain names (for example, [www.yourcustomdomain.com](#))

スクリーンショット 2023-10-21 21.53.01.png

## Domainを設定する。

「Add domain」を選択し、取得したドメインを入力して「Next」

(例：kanzen-ni-rikaishita.com)

#### SSL Certificates

SSL Certificates provide end-to-end encryption and integrity for all web requests to ensure information is transmitted securely.

[Automated Certificate Management \(ACM\)](#) is available for applications running on paid dynos to automate your SSL security.

Certificates acquired elsewhere may be configured using the [Manual Certificate](#) option.

#### SSL Certificates

[Configure SSL](#)

Your certificate is automatically managed. Add a custom domain to your app.

#### Domains

You can add custom domains to any Heroku app, then visit [Configuring DNS](#) to setup your DNS target.

Your app can be found at <https://runteg-festival-25666d1d1b53.herokuapp.com/>

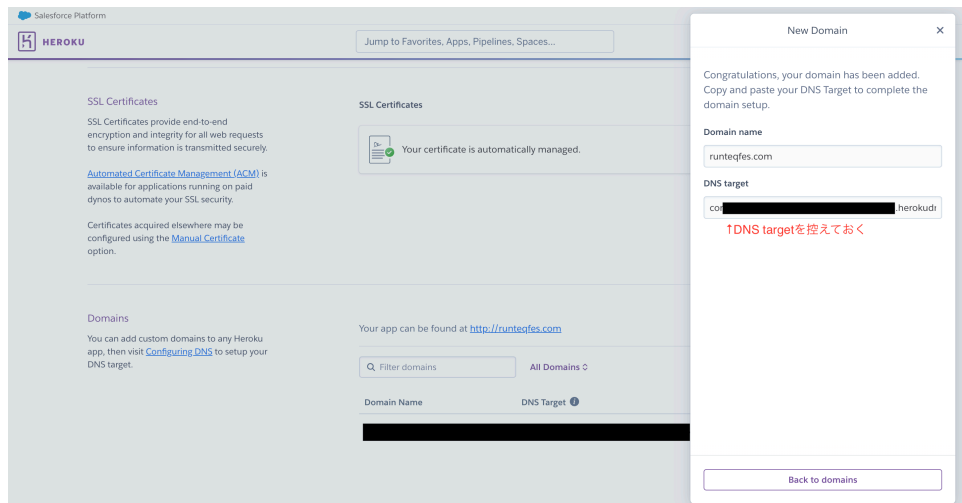
[Add domain](#)[All Domains](#)[Refresh ACM Status](#)

Custom domains will appear here

Custom domains allow you to access your app via one or more non-Heroku domain names (for example, [www.yourcustomdomain.com](#))

スクリーンショット 2023-10-21 21.53.01.png

この画面が出てくるため"DNS target"のURLを控えておく。

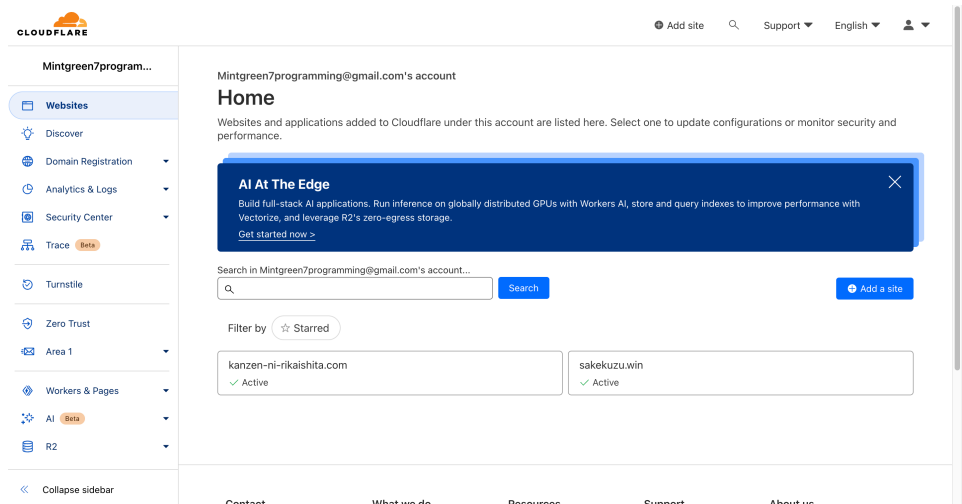


サブドメインが必要であれば、さらに「Add domain」で追加する。

(例：[www.kanzen-ni-rikaishita.com](http://www.kanzen-ni-rikaishita.com))

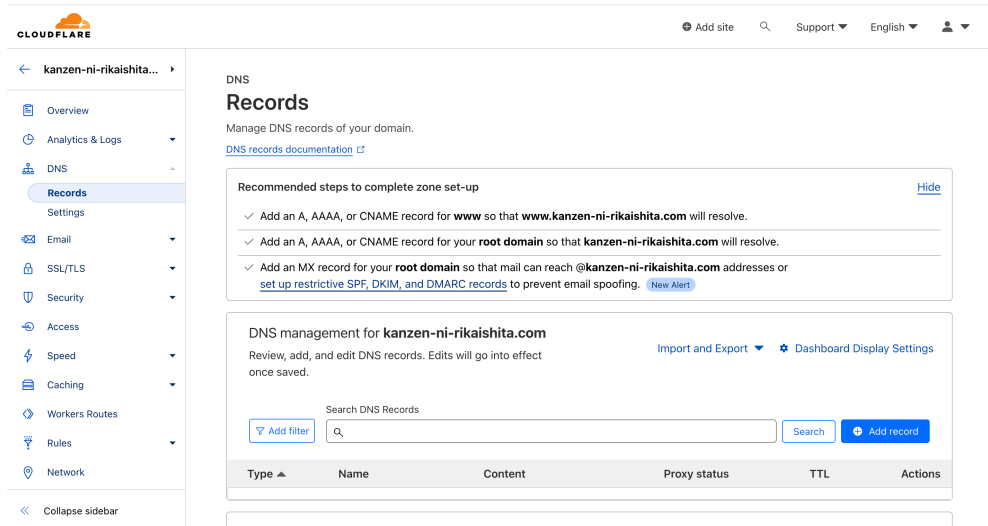
## ②Cloudflareで独自ドメインとherokuの発行したURLを紐付ける

CloudflareのHome(websites)から、設定したい独自ドメイン(ここでは、kanzen-ni-rikaishita.com)を選択する。

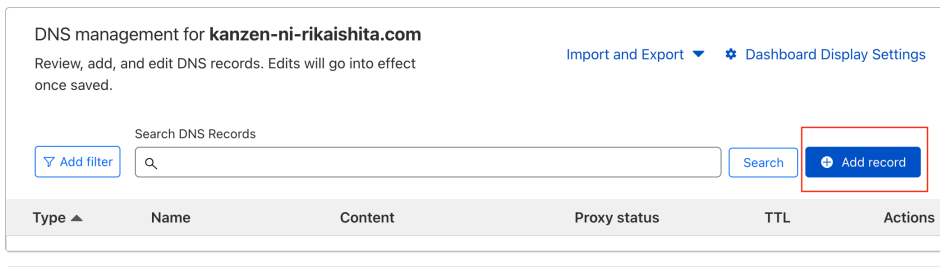


スクリーンショット 2023-10-21 21.56.09.png

## DNS > Recordsを選択する



### 「Add record」を押す



#### ● Typeに「CNAME」

CloudFlare では root もサブドメインも CNAME

<https://devcenter.heroku.com/ja/articles/custom-domains#configuring-dns-for-root-domains>

#### ● Nameにサブドメインを入力。

(サブドメインなし(root)の場合は「@」ここでは「www」を入力し、<https://kanzen-ni-rikaishita.com> でも <https://www.kanzen-ni-rikaishita.com> 両方でアクセスできるように2つ登録)

#### ● targetに heroku のDNS targetを入力し「save」

DNS management for **kanzen-ni-rikaishita.com**

Review, add, and edit DNS records. Edits will go into effect once saved. [Import and Export](#) [Dashboard Display Settings](#)

Search DNS Records

[Add filter](#)  [Search](#) [Add record](#)

**www.kanzen-ni-rikaishita.com** is an alias of **concave-garden-ejvtiyhegi19139x1pxq8l4m.herokudns.com** and has its traffic proxied through Cloudflare.

Type	Name (required)	Target (required)	Proxy status	TTL
CNAME	www	...com	<input checked="" type="checkbox"/> Proxied	Auto

Use @ for root **herokuのDomainで控えておいたDNS targetをtargetに入力**

**Record Attributes** [Documentation](#)

The information provided here will not impact DNS record resolution and is only meant for your reference.

Comment

スクリーンショット 2023-10-21 22.00.22.png

Proxy statusについては理解が浅いため間違いがあればご指摘ください。

onにするとCloudflare のグローバルネットワークを利用し、これによりキャッシュの最適化やより強固なセキュリティ保護を受けることができるようになりますが、DNSで登録されていたオリジンのIPアドレスではなく、Cloudflare の IP アドレスが返ってきます。

はてなブログなど IP アドレスの情報と紐づける場合は DNS only にしなければいけないようです。heroku は Proxied で設定できたのでこのままにしています。

参考：

<https://dev.classmethod.jp/articles/cloudflare-dns-basics-what-is-proxy/>

<https://blog-dry.com/entry/2023/07/12/071049>

参考：CNAME

<https://e-words.jp/w/CNAME%E3%83%AC%E3%82%B3%E3%83%BC%E3%83%89.html>

<https://www.cloudflare.com/ja-jp/learning/dns/dns-records/dns-cname-record/>

この状態ではまだ heroku 側に設定が反映されていません。  
※撮り忘れたので別のアプリの写真を使用しています。すみません

Domains

You can add custom domains to any Heroku app, then visit [Configuring DNS](#) to setup your DNS target.

You have multiple custom domains enabled Add domain

All Certificates All Domains Refresh ACM Status

Domain Name	DNS Target	ACM Status
sakekuzu.win		⌚ Waiting
www.sakekuzu.win		✔ Ok

アプリのターミナルで `$ heroku domains` と打ってください。

```
> Warning: heroku update available from 8.4.2 to 8.6.0.
=== boozefolio Heroku Domain

boozefolio.herokuapp.com

=== boozefolio Custom Domains

Domain Name DNS Record Type DNS Target

sakekuzu.win ALIAS or ANAME aaaaaaaaaaaaaafb.herokuapp.com abcdefgs...
www.sakekuzu.win CNAME daaaaaaaaaaaaafb.ns.com abcdefwww...
```

Domain Name が設定されていることを確認したら、

`$ heroku domains:wait` ドメイン名

例

```
boozefolio % heroku domains:wait sakekuzu.win
> Warning: heroku update available from 8.4.2 to 8.6.0.
Waiting for sakekuzu.win... done
```

Domains

You can add custom domains to any Heroku app, then visit [Configuring DNS](#) to setup your DNS target.

You have multiple custom domains enabled Add domain

All Certificates All Domains Refresh ACM Status

Domain Name	DNS Target	ACM Status
sakekuzu.win		✔ Ok
www.sakekuzu.win		✔ Ok

スクリーンショット 2023-10-23 11.50.26.png

これで完了です。

## ところで独自ドメイン is 何？

よく分からないまま取得してしまっていますが、改めて独自ドメインが何なのかも少しまとめてみたいと思います。

### 独自ドメインとは

- ・インターネット上のアプリの住所。サーバーの位置情報を示すIPアドレスを変換した文字列の部分。
- ・<https://runteq.jp> の場合、runteq.jp の部分がドメイン。ここが自分独自のもの、ということ。
- ・独自ドメインは戸建てを買うイメージ。自分だけ使える、自分のもの。一方で heroku など取得した URL は共有ドメインで、賃貸に入居してるイメージ。

<https://www.onamae.com/column/domain/50/>

[https://www.value-domain.com/media/only\\_domain/](https://www.value-domain.com/media/only_domain/)

### 独自ドメインを取得すると何がいの？

- ・URL が変わらない
- 共有ドメインの場合、サービス会社の終了・変更によってドメイン名が変更される可能性があります。
- ・信頼面
- ドメイン名に会社やお店の名前、サイト名を含ませることで認知度や信頼感を高めることができます。(詐欺サイトでこの面を利用されている、という側面もありますが...)
- ・SEO 効果がある
  - ・メールアドレスにも使える
- などがあります。

あとポートフォリオ的には必須とのご意見も

ドメイン・HTTPS 対応

この辺りから実践的な内容。

独自ドメインと HTTPS の対応は必須です。サーバは heroku でもいいですが、<https://t.co/7oVyybP5Oc> はポートフォリオ感丸出しなので絶対 NG です。独自ドメインを設定する際に DNS を正しく理解しておくとその後の作業や運用もスムーズとなります

す。

続く

— ひさじゅ@Web系転職に強いプログラミングスクール RUNTEQ (@hisaju01) [June 9, 2020](#)

[https://www.value-domain.com/media/only\\_domain/](https://www.value-domain.com/media/only_domain/)

<https://muumuu-domain.com/supports/about-domain>

## どこで取得できるの？

<https://jyn.jp/compare-domain-registrar/>

上記のブログ 忖度なしで書いてあり大変参考になりました。

- ・お名前.com(GMO)
- ・スタードメイン(ネットオウル)
- ・ゴンベドメイン(インターリンク)
- ・JPDirect(JPRS)
- ・Google Domains(Google) ⚠️ 現在サービス終了
- ・Cloudflare Registrar(Cloudflare)
- ・AWS Route53(Amazon)
- ・Gandi.net (Gandi)

の比較が載っています。

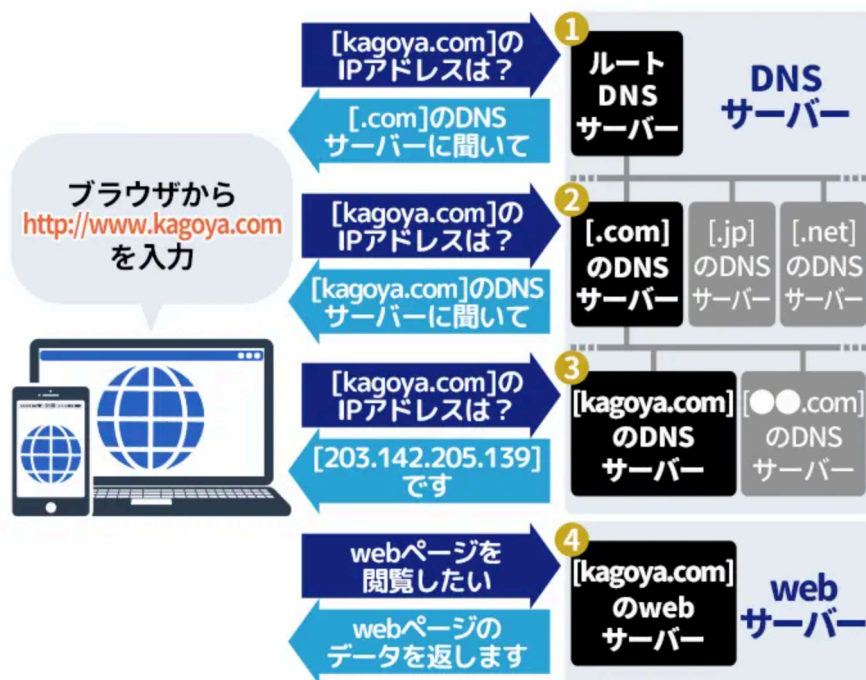
今回は、安全性を重視かつ個人サービスなので安さも求めたいという理由で Cloudflare を使用してみることになりました。

## 独自ドメインを取得する上で知っておくべきこと

### 1. ブラウザはIPアドレスに対してHTTPリクエストを行っている

ブラウザがURLをリクエストすると、以下の図のようにレスポンスを返します。





スクリーンショット 2023-10-23 13.12.30.png

- ①PCがDNSキャッシュサーバーに、ドメイン名でアクセスする
- ②DNSキャッシュサーバーがDNSルートサーバーに、管理情報ドメイン名に対応するIPアドレスは何かを問い合わせる
- ③DNSルートサーバーがDNSキャッシュサーバーに、管理情報のあるDNSサーバーを知らせる
- ④DNSキャッシュサーバーが管理情報のあるDNSサーバーに、ドメイン名に対応するIPアドレスを問い合わせる
- ⑤管理情報のあるDNSサーバーはDNSキャッシュサーバーに、IPアドレスを回答する
- ⑥DNSキャッシュサーバーはPCに、IPアドレスを回答する
- ⑦PCはIPアドレスでWebサイトにアクセスし、ブラウザ上にWebサイトが表示される

<https://www.kagoya.jp/howto/it-glossary/domain/web-01/>

<https://www.kagoya.jp/howto/it-glossary/domain/dns-server/> より引用

サブドメインに対するクエリの場合、さらに追加のネームサーバー(DNSサーバー)が追加され、そのネームサーバーはサブドメインのCNAMEレコードの保存します。

## 2. 独自ドメインにかかる費用は取得費だけでない

上記のように、独自ドメインを取得することはただの文字列の売買でなく、ドメインをネームサーバーに登録する必要があります。また、取得するだけでなく所有者の登録、情報の更新など管理も必要です。

Cloudflareなどはドメインレジストラと呼ばれ、このドメインとサーバーの設定や管理を行い、ドメイン名を購入・登録できるサービスを提供している組織です。当然、そのサービスや料金体制にはそれぞれ違いがあります。

Cloudflareの場合、この登録・更新費・管理費なども全て込みの年額で変動がないため分かりやすいのも良かったです。

他社では「初年度の登録費は無料だけど次年度からは更新に数千円かかる」「Whois登録は申し込まないとやってくれない」「ドメインの更新が自動でなく、期限を過ぎたら手数料が発生した」など色々と異なるので注意が必要です。

<https://qiita.com/minorun365/items/790635f211a14ad6b4db>

<https://domain-talk.net/domain-column/comb-whois/>

### 3. Whois 情報に個人情報が表示される

独自ドメインは戸建てを買うイメージと言いましたが、家を買うと登記するように、独自ドメインも誰がそのドメインを持っているかなどの情報が登録・公開されます。

#### 検索結果

#### kanzen-ni-rikaishita.com

```
Domain Name: KANZEN-NI-RIKAISHITA.COM
Registry Domain ID: 2823456826_DOMAIN_COM-VRSN
Registrar WHOIS Server: whois.cloudflare.com
Registrar URL: https://www.cloudflare.com
Updated Date: 2023-10-21T12:23:04Z
Creation Date: 2023-10-21T12:23:01Z
Registrar Registration Expiration Date: 2024-10-21T12:23:01Z
Registrar: Cloudflare, Inc.
Registrar IANA ID: 1910
Domain Status: clienttransferprohibited https://icann.org/epp#clienttransferprohibited
Domain Status: addperiod https://icann.org/epp#addperiod
Registry Registrant ID:
Registrant Name: DATA REDACTED
Registrant Organization: DATA REDACTED
Registrant Street: DATA REDACTED
Registrant City: DATA REDACTED
Registrant State/Province: Kagawa
Registrant Postal Code: DATA REDACTED
Registrant Country: JP
Registrant Phone: DATA REDACTED
Registrant Phone Ext: DATA REDACTED
Registrant Fax: DATA REDACTED
Registrant Fax Ext: DATA REDACTED
Registrant Email: https://domaincontact.cloudflare.com/kanzen-ni-rikaishita.com
Registry Admin ID:
Admin Name: DATA REDACTED
Admin Organization: DATA REDACTED
Admin Street: DATA REDACTED
Admin City: DATA REDACTED
Admin State/Province: DATA REDACTED
Admin Postal Code: DATA REDACTED
Admin Country: DATA REDACTED
Admin Phone: DATA REDACTED
Admin Phone Ext: DATA REDACTED
Admin Fax: DATA REDACTED
Admin Fax Ext: DATA REDACTED
Admin Email: https://domaincontact.cloudflare.com/kanzen-ni-rikaishita.com
```

(香川に住んでるのバレるやん)

お名前.comなどのサービスでは「Whois 情報公開代理」を代行するかどうか選択する必要がありますが、これを選択すると、whois 情報に表示される個人や会社の情報の代わりに、独自ドメイン取得に使ったドメイン会社の情報が表示されるようになります。Cloudflare の場合デフォルトで Whois 情報公開代理が付属しています。

ただし、ドメインの種類によっては登録者や連絡先などの公開が必須の場合もあります。

公開されているメールアドレスにドメインに関する警告や更新を煽るような内容のフィッシング詐欺メールが届くこともあるそうなので、自分の情報が公開されているかどうか知っておく必要があるかもしれません。

## 4.ドメインには保有期限があり、使用実績のあるドメインは、他のサイトからのリンク設定や検索エンジンの評価が残っている

<https://www.cybersecurity.metro.tokyo.lg.jp/security/guidebook/286/index.html>

何気なく取得したドメインが実は最近まで詐欺サイトとして使われていた、という可能性もあります。独自ドメインを手放したのちに、そういったリスクもあることを知っていないと思わぬトラブルに繋がりそうです。

## 5.SSL化で通信を暗号化する必要がある

独自ドメインを設定しただけでは、「このページは安全ではありません」等の警告がブラウザに表示されます。これは、HTTP接続のままだからです。HTTPは平文で情報をやり取りするため、暗証番号やクレジットカード番号などが第三者に傍受される可能性があります。SSLはデータの暗号化を行う仕組みで、これを防ぐものです。

簡単なサンプルアプリなら必要ないかもしれませんが、ログイン機能などがある場合はSSL化が必須かと思います。

=====

勉強していて私があまり意識できていなかったと感じた部分は以上です。

他にも注意点があればぜひコメント等で教えてください 🙏

## Cloudflareで取得した独自ドメインをHerokuで設定する方法を振り返る

ここまで書いてきて、改めてCloudflareで取得した独自ドメインをHerokuで設定するとき、何の作業を行っていたのか振り返りたいと思います(長い 😊)

- ・前提として、Herokuでデプロイしたアプリは、Herokuの共有ドメインが割り当てられる ([app name].herokuapp.com)。つまり、herokuapp.com および herokudns.com のネームサーバーへリクエストしてIPアドレスを探してWEBページを表示させている

- ・Herokuのsettingで「Domain」を追加すると「DNS target」が発行されネームサーバーに登録される。DNS targetは通常、ドメイン名を別のドメイン名に関連付けるCNAME（Canonical Name）レコードの値として使われる。このCNAMEレコードは、Herokuのアプリにアクセスするための一意の識別子となる

- ・DNS targetを使用してCloudflareで独自ドメインとHerokuドメインと関連付ける

- ・ `$ heroku domains:wait` で関連付けを反映させる
- ・「SSL Certificates」でSSLを設定する。

独自ドメイン、完全に理解した 😊

その他参考サイト

<https://blog.rmblankslash.net/entry/2021/09/21/080000>

[https://lpi.or.jp/lpic\\_all/linux/network/network07.shtml](https://lpi.or.jp/lpic_all/linux/network/network07.shtml)

## 未経験からフルリモートのエンジニアになってもうすぐ1年なのでフルリモ向いてる人を考えてみる

おかげさまで、未経験からエンジニアに転職してもうすぐ1年が経ちます。

今年お世話になった方々、改めて有難うございました。

<https://qiita.com/MIDO-ruby7/items/18ff8d14878a868b2005>

さてさて、私は転職当初からフルリモートで働いています。

フルリモで働いていることを伝えると、転職活動中の方や、内定先がフルリモートに決まった未経験エンジニアから「やっていけるでしょうか」「どんな人が向いていますか」と質問をもらうことも多いです。

そこで、今回は良く頂く質問に対して個人的見解をまとめてみようと思います。

つまりポエムです。

### 未経験者がフルリモートでエンジニアをやるのは可能ですか？

可能です。なぜなら、私も実際に未経験からフルリモートでエンジニアをやれているからです。

また、コロナ禍に就職されたスクールの先輩方は、皆さんフルリモートでエンジニア人生をスタートされています。コロナ禍終了後、出社されている方もいれば、そのままフルリモートで地方から仕事をされている方も多いです。

そういった方々にお話を伺うと、もちろん先輩に質問するタイミングが難しい、などの声はありましたが、コロナどうこうは関係なく、今から未経験で入るとしても同じように働けるだろうと言うお声が多かったです。

可能かどうかだけで言えば、可能です。

### 未経験者がフルリモートでエンジニアをやるのはやめた方がいいですか？

これは、2つの観点があります。

1. あなたがフルリモートに向いている人か
2. あなたの所属する企業がフルリモートに向いているか

私が企業について語るには、根拠や知識の足りない話になりそうなので、今回はどんな人がフルリモートに向いているかを考えてみたいと思います。

## 1.フルリモートに向いている人

### 報連相ができる

当たり前だと言われるかもしれませんが、実際にフルリモで働いていて、これが絶対条件だと個人的には思います。

出社する場合でも関係なく必要な能力ではありますが、他人の目が無いリモートでこれが無い場合、マネジメントやメンターの介入が遅れます。また、リモートはコミュニケーションの機会もリアルより大幅に減るため、一度すれ違いが起こると大きな問題やチーム内に支障が起こりやすいです。

例えば弊社の場合、業務について報告と相談をするのは朝 10 時から 30 分程度のミーティングだけです。全員の issue の進捗状況や困っていることが無いかを確認します。

それ以外は基本的に一人で実装を進めます。

さて、その間に困ったこと・詰まっていることがあったとき、あなたはどんな行動を取るでしょうか。

もしくは、自分の実装によるインシデントを発見したときはどうでしょうか。

自分ではなく、メンバーが困っていたらどうでしょうか。

YouTube が見なくなったときはどうでしょうか。

他のメンバーはどうしているのでしょうか。

何を誰に報連相すれば良いのでしょうか。

周りを見渡しても誰もいません。唯一、Slack や Google カレンダーでメンバーの今日の予定を察することができます。

あなたは家に一人のとき、どんな行動を取って、何を優先するでしょうか。

.....なんだか、不安になりませんでしたか？

今あなたが感じた「なんだか不安」という不安感は、働き始めたあなたに対してのメンバーやマネージャーの最初の評価です。

この不安感の原因には、業務についてよく知らない、自分の力量が分からない、などさまざまな理由があります。しかし、一番根本的な原因となる問題は、あなたとメンバーがお互いについてを良く知らないことです。

未経験から転職して働き始めたとき、あなたやメンバーがどんな人で、何を考えて、どう行動するのかを理解することが、最初のハードルになります。

リモートは、この相手との信頼関係を築くコミュニケーションの機会が圧倒的に少ないです。人間は、顔を見て今日の天気の話をしただけでも相手への緊張感は下がり理解度が上がります。

「報連相ができる」と一言で言いましたが、リモートの場合は、この機会を自ら作り、適切にコミュニケーションや会話をする必要があるのです。自分の考えをきちんと伝え、素直に困ったこと・嬉しかったこと・やりたいことなどを開示し、自分からコミュニケーションを取れる人が「報連相ができる」人です。

「サポートは必要だが、この人なら放っておいても安心」とメンバーから信頼される努力ができない人は、リモートに向いてないと思います。

新卒がフルリモートに向かないのは、この周囲の安心感を得るのがどうしても最初は難しいこと、仕事上のやり取りが身につく前にフルリモートをする事で失敗を相談できなかったり孤立するからです。

逆に言えば、新卒でもバイトやインターン、研究室、部活などの経験からこれが備わっている人は最初からフルリモートできるのではないかなと思います。

前職が営業だった経験から、転職など新しい集団に入った最初の1ヶ月にどんなことすれば良いのかをまとめた記事があります(タイトルはRUNTEQについてですがその中で"組織で必要とされる人材になる行動指針"について考えた記事です)。

お互いについてを良く知らないことがなぜハードルになるのか、具体的にどんな行動をすれば良いのか参考に知りたい方は覗いてみてください。

[https://note.com/mintgreen\\_mido/n/n7a78982b9840#5256EBCD-5FCC-4742-8BA6-29661839B18B](https://note.com/mintgreen_mido/n/n7a78982b9840#5256EBCD-5FCC-4742-8BA6-29661839B18B)

## 自分で自分をコントロールできる

これも、社会人として必要な力かと思いますがフルリモートの場合はもっと切実に健康面でこれが必要だと実感しています。

一日中家にいるので、自分の健康状態・精神状態を把握しづらいのです。

人と話していてイライラする

今日はなんだか歩くと息切れする

前向きな気持ちになれない



日常の中で、些細な身体サインがあるかと思いますが外に出たり人と接する時間が少ないのでこのサインを見過ごしがちです。

また、外に出ないことで、だんだんと慢性的な疲労感や疲れを感じ始めます。やばい。

さらに言えば、外に出ていればスタバでマックしたり、マクドでマックしたり、色々と気分転換の機会がありますが、フルリモートだとその機会も意識的に取らないといけません。

- 自分がどんな状態の時に疲れているのか知っている
- 自分が疲れている時の気分転換の方法を知っている

かどうかは非常に重要だと思いました。多分、これが下手だと長期的に病むか・死にます。私は身の危険を感じております。

## 余談

最後にする話ではないのですが、フルリモートで働いてみて見つけた健康面のメリットは**自由に気兼ねなくトイレに行けるのでお通じが良くなった**ことです。便秘の人にはおすすめです。共感してくれる人がいると嬉しいです。

フルリモ向いてる要素は、他にも色々あると思いますが、私が実感しているフルリモート1年目の感想は、特に上記2点かなと思います。

皆さんが思う、フルリモートに向いている資質があればぜひコメントで教えてください。

## ChatGPT の効果的な使い方

# Samlpe Title

## ChatGPT の効果的な使い方

**Samlpe Title**

## JavaScriptからの護身術

JavaScriptを書くのってめんどくさいですか。

フロントエンドのライブラリやフレームワークを使わない開発をするときなんか特に。

例えばRuby on RailsでJavaScriptをゴリゴリと書かざるを得ないって状況に遭遇したら、

「Rails使ってたからRuby以外書きたくない。」

って、私は思ってしまうものです。

また、私の周りにはJavaScriptが苦手であったり、「好きじゃない」という人もいます。

それでもJavaScriptというものは、頻繁に開発者の目の前に現れて、

「さあ、私を書け！それを実装するには私が必要だぞ！さあ、私を書け！」

と迫ってきます。

そこで使えるのが、JavaScriptから身を守る術——「JavaScriptからの護身術」です。

ということで「JavaScriptのことを好きになれないし、書きたくない」という人に向けて、JavaScriptが必要  
そんな実装を、JavaScriptを使わずに実装する術を伝授します。

では、JavaScriptからの護身術の稽古を始めましょう。

## JavaScriptからの護身術のデモンストレーション

<https://js-defense.botofune.com>

上記のページで、以下の9種類のJavaScriptからの護身術のデモンストレーションを体験できます。

1. ドロワーメニュー
2. モーダルダイアログ
3. ドロップダウン
4. フィルタリング
5. リアルタイムバリデーション
6. SPA 的ページ遷移アニメーション
7. オンボーディングツアー
8. トースト
9. スライダー

これらはすべてHTMLとCSSだけで実装しており、JavaScriptから身を守っています。

コードの細かい内容は載せませんが、デベロッパーツールで実装の内容を確認できるので、ぜひ覗いてみて  
ください。

以降は各実装のポイントを解説します。

## ドロワーメニュー

ボタンを押したら、横からヌルっと出てくるようなアレです。

よくある実装方法の例として、ドロワーメニューを開くボタンを押した際に、JavaScriptでドロワーメニューを表示するクラスを付与する方法が挙げられます。

そしてその方法で立ちのけるJavaScriptは、`<input type="checkbox">`で退けることができます。

アニメーションが不要な場合、実装に必要な最低限のコードは以下の通りです。

```
<!-- ドロワーメニューを開くボタン（本当はボタンじゃないけど） -->
<input type="checkbox" />

<!-- ドロワーメニュー -->
<div>
 ドロワーメニュー
</div>
```

```
div {
 /* デフォルトでドロワーメニューを非表示 */
 display: none;
}

/* <input type="checkbox" />にチェックが入っている場合、 */
input:checked ~ div {
 /* ドロワーメニューを表示 */
 display: block;
}
```

もう気づきましたか。

`<input type="checkbox">`と`input:checked {}`を使えば、なんだか色々できそうだと。その通りです。

このチェックボックスを使った技"チェックボ拳"こそが、JavaScriptから身を護るための必殺技です。

## モーダルダイアログ

ドロワーメニューと並ぶUI界のスーパースターであるモーダルダイアログの実装においても、JavaScriptから身を護ることができます。

```
<!-- モーダルダイアログを開くボタン（本当はボタンじゃないけど） -->
<input type="checkbox" />

<!-- モーダルダイアログ -->
<div>
 モーダルダイアログ
</div>
```

```
div {
 /* デフォルトでモーダルダイアログを非表示 */
 display: none;
}

/* <input type="checkbox" />にチェックが入っている場合、 */
input:checked ~ div {
 /* モーダルダイアログを表示 */
 display: block;
}
```

もう気づきましたか。

ドロワーメニューの実装と全く同じだということを。

その通りです。

"チェックボ拳"を使えば、ドロワーメニューもモーダルダイアログも対応できます。

上記のコードには書いていないですが、"表示する位置が画面の端か中央か"ぐらいの違いしかありません。

ただしドロワーメニューと比較すると、モーダルダイアログのこの実装はHTMLのセマンティックさに欠けると感じる人がいるかもしれません。

HTMLには `<dialog>` という、名前からしてモーダルダイアログの実装にピッタリっぽいタグがあります。

`<dialog>` の主要な機能として以下が挙げられます。

- 例えば `z-index: 9999`; の要素があったとしてもダイアログの方が前面に表示される
- ダイアログを開いたときにダイアログ内のフォーカス可能な要素にフォーカスする
- ダイアログが開いている状態でEscキーを押すとダイアログが閉じる

このように機能が豊富な `<dialog>` ですが、これらを扱うにはJavaScriptが必要になります。

多少のJavaScriptとの接触に耐えられる自信がある人は、`<dialog>` を使用するとよいでしょう。

「あらゆるJavaScriptを退けるぞ!」という信念をお持ちの人は、"チェックボ拳"を使いましょう。

## ドロップダウン

HTMLの `<details>` を使えば、めちゃんこシンプルにドロップダウンを実装できます。

```
<details>
 <!-- ドロップダウンの開閉ボタン -->
 <summary>ドロップダウンを開く</summary>
 <!-- ドロップダウンの中身 -->
 <div>メニュー1</div>
</details>
```

```
details {
 position: relative;
}
```



```
div {
 position: absolute;
 top: 100%;
 left: 0;
}
```

`<details>` はいわゆるアコーディオンメニューの実装に最適なタグかと思いますが、アコーディオンを開いたときに表示する要素を浮かせるだけで、ドロップダウンを実装できます。

ただし、ドロップダウンの要素を非表示にする際のアニメーションは、2025年9月時点では一部のブラウザでしか実装できません。

ドロップダウンを閉じるときのアニメーションがどうしても今すぐ必要な人は、唇を噛み締めながらJavaScriptを使用しましょう。

あと"おま環"か実装の問題かもですけども、デモンストレーションの実装だとアニメーションが発生しないときがあるので、動作の安定性に欠けます。

## フィルタリング

これは"チェックボ拳"と兄弟的な関係の技"ラジオ拳"を使うことで、JavaScriptから身を護ることができます。

```
<input type="radio" name="filter" class="filter1">
<input type="radio" name="filter" class="filter2">
<p class="group1">グループ1</p>
<p class="group2">グループ2</p>
```

```
/* filter1のラジオボタンにチェックが入っている場合、 */
/* group1を表示 */
.filter1:checked ~ .group1 {
 display: block;
}
/* group2を非表示 */
.filter1:checked ~ .group2 {
 display: none;
}

/* filter2のラジオボタンにチェックが入っている場合、 */
/* group1を非表示 */
.filter2:checked ~ .group1 {
 display: none;
}
/* group2を表示 */
.filter2:checked ~ .group2 {
 display: block;
}
```

"チェックボ拳"と大きく異なる点は、inputのtypeが"checkbox"ではなく"radio"なところぐらいです。あとは"チェックボ拳"と同じ要領で、"ラジオ拳"を繰り出すことができます。

なお、デモンストレーションの実装は単一の条件で絞りたい場合の実装ですが、複数の条件で絞りたい場合は"チェック拳"を使いましょう。

## リアルタイムバリデーション

リアルタイムでユーザーの入力を監視しなければならないので、JavaScriptが必要そうだと感じますよね。

大丈夫です。

JavaScriptから身を護りつつユーザーの入力を監視して、ただちにバリデーションのメッセージを表示することができます。

```
<input type="text" pattern="{1,10}" placeholder=" " required>
<p>あかん</p>
```

```
/* バリデーションのメッセージはデフォルトで非表示 */
p {
 display: none;
}

/* バリデーションに引っ掛かったら、メッセージを表示 */
input:not(:placeholder-shown):invalid + p {
 display: block;
}
```

input に `pattern="{1,10}"` と書くことで、"入力値が 1 ～ 10 文字ではない"場合、CSS の `:invalid` が適用されます。

ただし初回表示時は何も入力していない状態なので、入力値が1文字未満としてバリデーションに引っ掛かってしまいます。

ということで、バリデーションのメッセージの表示条件として `:not(:placeholder-shown)` を設定しています。

これによりバリデーションのメッセージを表示する条件が、"プレースホルダーが見えていないとき（つまり何かしら入力されているとき）、かつ pattern にマッチしない場合"になります。

そしてinputに `placeholder=" "` を設定することで、初回表示時は"patternにはマッチしないが、プレースホルダーが見えている状態"になるので、バリデーションのメッセージが表示されないわけです。

とはいえ10文字を超えて入力されたくない場合は、inputに `maxLength="10"` を設定することで対応できます。

10文字を超えて入力されたくないのであれば、入力できる文字数の上限を10文字にすればいいのです。

このリアルタイムバリデーションの活用方法はあんまり無さそうですが、例えば"以前はユーザー名は20文

字まで入力可能だったが、仕様変更により10文字までになった"みたいなケースだと生きそうです。  
 この場合、例えばユーザーが自身の名前を編集できるページに遷移した直後にはバリデーションメッセージが表示されているので、仕様が変わったことをユーザーに知覚させやすいです。

## SPA 的ページ遷移アニメーション

SPA 的ページ遷移というのは、JavaScript で DOM とか URL とかを变える方式のページ遷移を指します。

「JavaScript から身を護ると言っておきながら、ページ遷移において JavaScript の介入をゆるすのか」という JS 護身流過激派からの声もありますが、

「利用するライブラリやフレームワークの都合上、仕方がない場合もあるものです」

という言葉を読み込みつつ、謝罪をいたします。

それでも仕方なしに SPA 的ページ遷移を使わざるを得ない状況においても、諦めて全ての JavaScript を受け入れる必要はありません。

例えばページの遷移に伴うアニメーションは、HTML と CSS で対応することがある場合があります。

以下は"すごいページだけ背景を黒くしたい"場合に、滑らかに背景色を変更する例です。

```

{ /* すごくないページ */ }
<div className="wrap">
 <Link href="/sugoi">
 すごいページへ行く
 </Link>
</div>

```

```

{ /* すごいページ */ }
<div className="wrap sugoi">
 <Link href="/sugokunai">
 すごくないページへ行く
 </Link>
</div>

```

```

body {
 transition: background 1s;
}

/* bodyがクラスsugoiの要素を持つ場合、背景を黒くする */
body:has(.sugoi) {
 background: black;
 color: white;
}

```

`body:has(.sugoi)` によって、クラス `sugoi` の要素があるページに遷移すると背景が黒くなります。

"すごいページ"は `<div className="wrap sugoi">` と書いてある通りクラス `sugoi` の要素があるので、"すごいページ"に遷移すると背景が滑らかに黒くなります。

ちなみに `:has()` は、前述の"チェックボ拳"と"ラジオ拳"の弱点を補うことができます。

実は今まで隠していたのですが、"チェックボ拳"と"ラジオ拳"には弱点があります。

チェックボックスやラジオボタンが、スタイルを変えたい要素と同階層か親階層にいないと、"チェックボ拳"と"ラジオ拳"は発動できないのです。

あらためて、ドロワーメニューを実装する場合の"チェックボ拳"を確認してみましょう。

```
<input type="checkbox" />
<div>
 ドロワーメニュー
</div>
```

```
div {
 display: none;
}
input:checked ~ div {
 display: block;
}
```

"チェックボ拳"の肝は `input:checked ~ div` です。

これは、"チェックボックスにチェックが入ったら、チェックボックス以降の同階層の div が対象"という意味です。

つまり以下のように input と div が同階層に無い場合は、"チェックボ拳"を発動できません。

```
<p>
 <!-- input:checked ~ divの対象となるdivが同階層に無い -->
 <input type="checkbox" />
</p>
<div>
 ドロワーメニュー
</div>
```

この弱点により、例えば"チェックボックスが複数ある場合に、それらを flex や grid で横並びに配置したい"といった場合は、"チェックボ拳"を使えない可能性があります。

そんな絶体絶命な"チェックボ拳"ですが `:has()` との合わせ技で、さきほどの状況でも"チェックボ拳"を発動できるようになります。

```
<p>
 <input type="checkbox" />
</p>
<div>
 ドロワーメニュー
</div>
```

```
div {
 display: none;
}
/* pがinput:checkedな要素を持っていたら、p以降のdivを表示する */
p:has(input:checked) ~ div {
 display: block;
}
```

このように `:has()` と組み合わせれば、大抵の状況で"チェック拳"や"ラジオ拳"を繰り出すことができます。

とはいえ `:has()` にも弱点がありまして、それは"パフォーマンス性を損なうかもしれない"ことです。くわしくは"`:has performance`"とか"`:has パフォーマンス`"とかでググってください。

`:has()` は挙動から分かる通り、要素の状態を監視しています。

そして監視対象の変化に応じて、スタイルを適用する処理を行ないます。

このように捉えると、「他のCSSプロパティに比べて重そうな処理をしうだな～」と想像できますね。なので監視対象はできるだけ狭い範囲にした方がよさげです。

`html:has(div)` みたいな「全 div を監視したるで！」ではなく、`.button:has(.text)` のように「すみません、ほんとココだけ、ココだけの監視を、何卒、、、」みたいな。

## オンボーディングツアー

サービスにアカウントを登録すると、「まずはアカウント情報を更新しよう」みたいな案内が出て、それを済ませると次は「投稿してみよう」みたいな、使い方を次々と案内してくれるサービスに遭遇したことはないですかね。

ここでいうオンボーディングツアーとは、そういう使い方をガイドしていくヤツのことです。

デモンストレーションでは、新技"ターゲット拳"を使用しています。

```
スタート
次のステップ
最初から
```

```
a:target::after {
 content: "←いまここ";
}
```

`:target` は URL に含まれる `#` の値と一致する `id` の値を持った要素に、スタイルを適用する疑似クラスです。

例えば `<div id="example">` という要素があり、URL が `example.com#example` になると、先述の要素にだけ CSS を適用することができます。

ちなみに"ラジオ拳"でも、オンボーディングツアーを実装することが可能です。

ただし、"ラジオ拳"に比べて"ターゲット拳"の方がHTMLの構造に制約が少ないです。

オンボーディングツアーはコンポーネントを横断的に実装しなければならないケースが多い気がするので、"ターゲット拳"の方が向いていると考えられます。

そんな"ターゲット拳"には、以下の副作用があります。

- URLに # が付く
- ブラウザのセッションの履歴に残る

上記の副作用が人や場合によっては、弱点になるかもしれません。

"ターゲット拳"を使用するには、"URLに # を付け加えなければならない"という制約があります。

この制約が人によっては、「URLが変になった」や「URLに変なものを付けるな」といったネガティブな反応になることが想定できます。

付随して、ブラウザの 戻る や 進む の挙動も変わります。

試しに前述のデモンストレーションのオンボーディングツアーでステップ2まで進んだ後に、ブラウザの 戻る を実行してみましょう。

おそらくステップ1がハイライトされます。

この現象も人によっては、「前のページに戻ると思ったのに、オンボーディングから抜け出せない」といった不満が出てくることが考えられます。

とはいえステップの進捗をURLで表すことができるので、ステップが長い場合は有効な手段になり得るかもしれません。

また、スマホで 戻る の操作をスワイプで行なうことに慣れている人、かつ「ひとつ前のステップをやり直したい」といった場合は、履歴に残っている方がよさそうです。

"ターゲット拳"は、使いどころの難しい技と言えそうですね。

## トースト

ユーザーがサインインしたときに画面の端っから「サインインしました」みたいなメッセージが出たり、フォームをサブミットしたときに「送信しました」みたいなメッセージが出るサービスあるじゃないですか。

トーストとはそれです。

以下はページ表示直後にトーストを表示する例です。

```
<div>トースト</div>
```

```
@keyframes toast {
 5%, 95% {
 translate: 0;
 }
}
```

```

100% {
 translate: calc(100% + 1rem);
}
}
div {
 position: fixed;
 top: 1rem;
 right: 1rem;
 translate: calc(100% + 1rem);
 animation: toast 3s;
}

```

トーストを画面外に配置しておき、`animation` で画面内に表示して、最後に画面の外へ移動させています。

ちなみに、要素を動かすときは `top` や `right` などの `position` プロパティの値ではなく、`transform` 系を使った方がよい場合が多いです。

ブラウザはHTMLとCSSを表示するまでに複数の段階を踏みますが、ブラウザとCSSのプロパティによって、どの段階から処理をするかが異なります。

ブラウザごととCSSのプロパティごとの差異は、以下のページが参考になります。

<https://lab.skk.moe/css-triggers>

上記のページによると、`position` 系の値はどのブラウザも `Layout` から処理しますが、`transform` はChromeとFirefoxにおいては `Composite` だけの処理で済みます。

## スライダー

スライダーを実装するとき、問答無用でSwiperやslickを導入していませんか。

でも「ラジオ拳」を使えば、あら不思議、ライブラリ使わずともスライダーを（ある程度）実装できます！

```

<input type="radio" id="slide1" name="slide" checked>
<input type="radio" id="slide2" name="slide">

 1
 2


```

```

ul {
 overflow: hidden;
 display: flex;
}

li {
 width: 100%;
 flex-shrink: 0;
}

```

```
#slide2:checked ~ ul li:nth-of-type(2) {
 translate: -100%;
}
```

自動でスライドしないスライダーであれば、このようにかなりシンプルに実装できます。

対してデモンストレーションに実装してある、次のスライドまたは前のスライドを表示するボタンの設置はそこめんどいです。

以下はデモンストレーションのコードの抜粋です。

```
<!-- 次のスライドを表示するボタン -->
<label for="slide3">
 <
</label>
<label for="slide1">
 <
</label>
<label for="slide2">
 <
</label>

<!-- 前のスライドを表示するボタン -->
<label for="slide2">
 >
</label>
<label for="slide3">
 >
</label>
<label for="slide1">
 >
</label>
```

表示中のスライドごとに、“前へ”と“次へ”のボタンを入れ替えることで実装しています。

スライドの1枚目が表示されている場合、各ボタンの挙動は以下になります。

- “前へ”を押された場合: スライドの3枚目を表示
- “次へ”を押された場合: スライドの2枚目を表示

スライドの2枚目が表示されている場合、各ボタンの挙動は以下になります。

- “前へ”を押された場合: スライドの1枚目を表示
- “次へ”を押された場合: スライドの3枚目を表示

このように、表示中のスライドによって“前へ”と“次へ”の挙動が変わるので、挙動ごとにHTMLとCSSを書く必要があります。



また、CSSの `animation` を使用することで、自動でスライドさせることができます（やったことないのでたぶん）。

ただし、`<input type="radio">` の `checked` と同期させることはできないでしょうけども。

自動でスライドさせつつ、アクティブなスライドを手動で切り替えられるようにしたい場合は、残念ながらJavaScriptを使うほかありません。

人生はスライダーです。

人生をスライドしていく中で、時には己の信念をスライドしなければならないこともあります。

## JavaScriptからの護身術の最終奥義

これまでに紹介したJavaScriptからの護身術で実装されたコードのレビューをお願いされたら、私はおそらくApproveしないでしょう。

セマンティックなマークアップになってなさそうだし、それに伴って余計なCSSが書かれていそうなので。

これまでに紹介したJavaScriptからの護身術は、JavaScriptが伸ばさなければいけないシワを、HTMLとCSSに押し付けているに過ぎません。

結局は誰かがアイロンを掛けて、シワを伸ばさなければならない。

ならば自分が伸ばさなければいけなかったシワは、自分で伸ばすのがスジってもんでしょうよ。

最後に、JavaScriptからの護身術の最終奥義をお伝えします。

合気道の達人、塩田剛三の言葉を紹介します。

弟子「合気道の最強の技ってなんですか」

塩田剛三「相手と友だちになることだぜ」

JavaScriptからの護身術の最終奥義とはすなわち、"JavaScriptと友だちになること"です。

友だちから身を守る必要は無いですからね。

以上でJavaScriptからの護身術の稽古を終わります。

JavaScriptと握手！

礼！

解散！

## おわりに

最後までお読みいただき、ありがとうございました。

本誌の制作は、多くの寄稿者と技術コミュニティの協力によって実現しました。

感想やフィードバックは、次回の制作にも活かしていきたいと考えています。

X（旧 Twitter）で `#RUNNEST_Hub` を付けて投稿いただけると嬉しいです。

## 奥付